

```
poster.yaml

type: poster
headline:
  "By design."
accent: #2B4AF2
layout: editorial

# the model writes intent
```

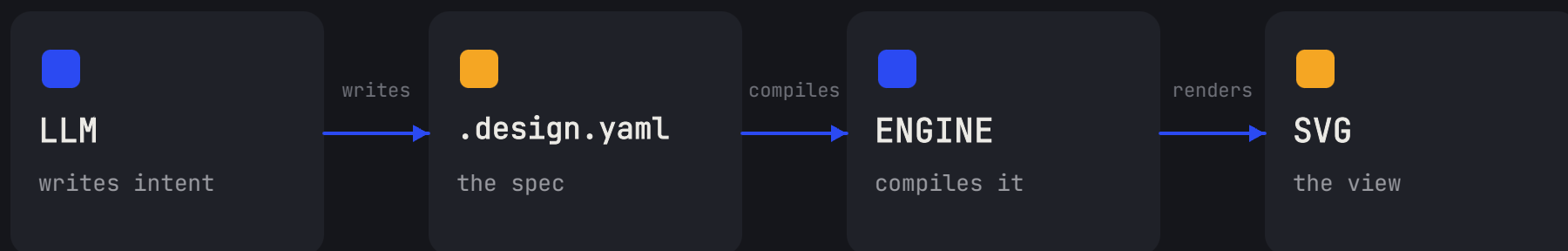
By design.

The engineering decisions behind a local-first,
model-native design engine.

SWIPE →

A compiler, not a chatbot.

Folio doesn't "draw". The model writes intent as a spec, the engine compiles it into a render tree, and the file on disk is the source of truth. Each part does exactly one job.



The model reaches the engine over MCP. The editor opens the same .design.yaml for humans.

The design file is the product. Everything else is a view.

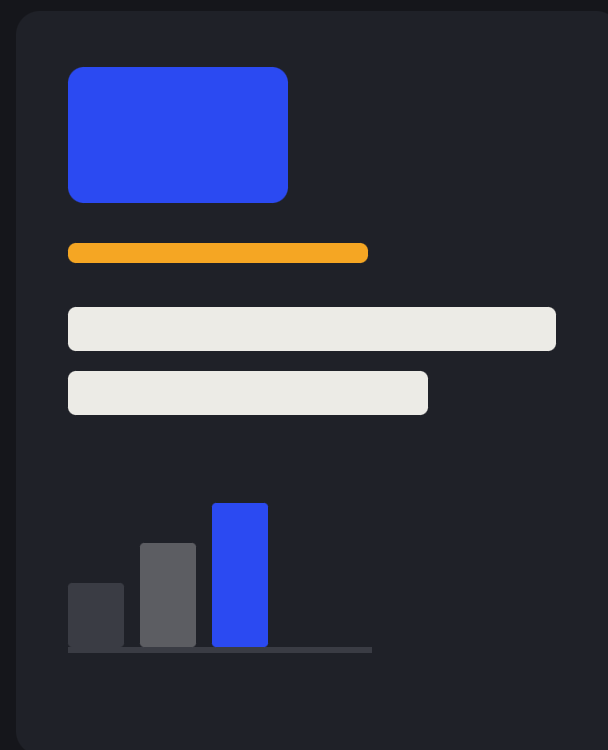
Design is structure, not pixels.

A layout is geometry — grids, ratios, spacing, hierarchy. A text-only model can reason about that and emit it as a spec. So even a small model running in LM Studio produces real designs.

```
model.yaml

rect    pos:[0,0,1080,1350]
text    pos:[90,120]  "Title"
text    pos:[90,300]  "73%"
bars    pos:[90,600]  62,31,18
donut    pos:[90,900]  40,35,25

# geometry — numbers, not pixels
```



No vision required.

The engine validates the math, fits the canvas, and fixes collisions.

YAML, not JSON. Not TOON.

The spec is written and read by models and humans. YAML wins both: every model knows it from training, it diffs cleanly line-by-line, and it takes comments — JSON takes none.

layer.json

```
{  
  "type": "text",  
  "pos": [90, 120],  
  "value": "Title",  
  "size": 72  
}
```

≈ 38 tokens

layer.yaml

```
text:  
  pos: [90, 120]  
  value: Title  
  size: 72  
  
# comments welcome
```

≈ 22 tokens • shorthand goes -80%

TOON is leaner still — but models and people don't speak it yet.
Ubiquity beats a marginal token win.

Vectors, not pixels.

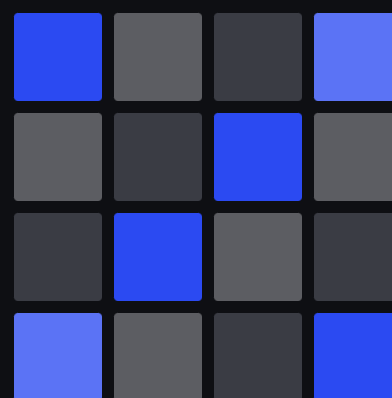
Folio renders SVG, not a bitmap canvas. The output IS the spec: resolution-independent, text stays selectable, files stay tiny, and it edits and animates losslessly.

VECTOR • SVG



smooth — scales to any size

BITMAP • PNG



fixed — locked to one resolution

Resolution-independent. Selectable text. Kilobytes, not megabytes.

And a true vector PDF falls out for free.

One engine. Every surface.

The engine speaks MCP — a standard tool protocol. Write it once, and the same surface works everywhere a model lives.



A standard protocol in. Every client out — 21 tools, one surface.

Bun. No build step.

The servers run straight from TypeScript source with `bun --smol` — no bundling, no transpile wait. Cold start under a second, a smaller footprint, and deploys are a copy and a restart.

```
~/folio

$ bun --smol src/mcp/http-server.ts

▸ folio mcp :3333 ready in 0.42s

▸ editor :4173 ready

# no build • no bundler • just run
```

< 1s

COLD START

-20%

MEMORY • --smol

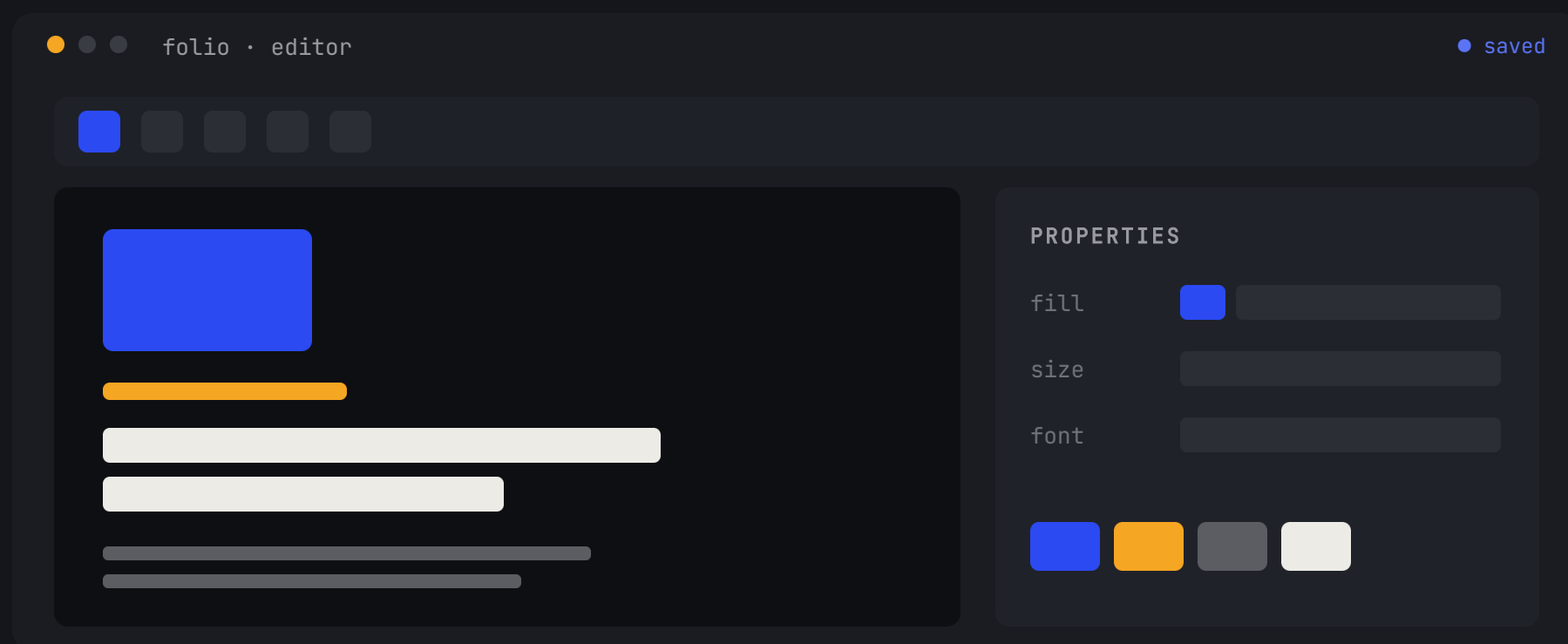
~4s

DEPLOY • cp + restart

The bundler never touches the hot path — Vite builds only the editor dist.

We built the editor too.

Generation gets you 90%. The last 10% is taste — a human call. So the same file opens in a visual editor: drag, recolor, retype. Non-devs never see code; devs drop to the YAML.



The file is the contract between model, human, and renderer.

Built for models and humans.

Every layer chosen for one job: model intent in, crisp editable design out.

LANG

TypeScript · strict

RUNTIME

Bun --smol

RENDER

SVG-in-HTML

DATA

YAML · js-yaml

UI

Vanilla TS · no framework

PROTOCOL

MCP · 21 tools

EXPORT

resvg · jsPDF · vector

TESTS

Vitest · Playwright

407 .ts files · 3000+ tests · <1s cold start · <100ms first render

GENERATED WITH CLAUDE OPUS 4.8 + FOLIO

github.com/azzindani/Folio →